CS5187 · Selected project presentation · 10 Nov 2026

LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

A fuzzer that writes valid SQL, checks the answers, and learns new SQL features from PostgreSQL's own commits.

Venu Kummari · **Bhuvan Chilukuti** · **Gowtham Reddy**

9 bugs reported **6** fixed upstream

Our target: PostgreSQL

- Widely used open-source database: banks, companies, apps
- The query planner changes almost every week, and new code is the least tested code
- We fuzzed the development branch (master) with internal checks on, and compared it with release 18

What we count as a bug

Crash	the server process dies or an internal assertion fails
Wrong result	the query runs fine but returns the wrong rows
Internal error	a valid query fails with "this should never happen"
Hang	the query never finishes and cannot be cancelled

9 bugs reported to PostgreSQL

6 fixed and committed upstream

5 of those fixes are our patches

How our fuzzer works

1

Learn

An LLM reads recent PostgreSQL commits and proposes new grammar rules; a gate keeps only rules the server runs

2

Generate

Typed SQL grammar (78 rules, 421 alternatives) builds queries that are valid by construction

3

Run

On master with assertions on, under 18 planner settings, and on release 18.6

4

Check

Oracles decide whether the answer is wrong (below)

5

Reduce & report

Shrink the query to a few lines and draft the bug report

The checks (oracles): how we know an answer is wrong

Crash check

server dies or an assertion fails

Rewrite (TLP)

split WHERE into true / false / NULL parts; they must add up

Plan variants

different planner settings must give the same rows

Cross-version

master and release 18 must agree

Hang check

a query must stop at the timeout

Plus small targeted fuzzers for new features (e.g. extended statistics).

Bugs 1–2: a planner crash and a broken to_char()

01

FIXED · #19803

Parallel planner crash

The planner builds a parallel plan for a table, then marks the table "not parallel-safe" but keeps the plan it already built. An assertion stops the server.

```
CREATE TABLE pi (a int);
CREATE TABLE pi_child () INHERITS (pi);
WITH c1 AS (SELECT 1 FROM pi WHERE true
            UNION ALL
            SELECT 1 FROM pi WHERE false),
c2 AS MATERIALIZED (SELECT 1)
SELECT 1 FROM c1 UNION ALL SELECT 2 FROM c2;
-- server crashes (assertion failure)
```

Found by the LLM arm: grammar rules learned from a recent planner commit produced this query; the crash check caught it.

Fix David Rowley, commit abda2a2, back-patched to v14.

02

FIXED · OUR PATCH · #19771

to_char() drops part of its output

When a number format ends with a decimal point, the end-of-string byte is copied into the output, so everything after it disappears. No error.

```
SELECT to_char(100::numeric,
               'FM999.SG "items"');

-- expected:  100+ items
-- got:       100
```

Found by cross-version check: thousands of random to_char calls on master and release 18, answers compared.

Fix our patch, committed by Tom Lane (776df8a) for v19.

Bugs 3–4: a statistics crash and a wrong NOT IN

03

FIXED · OUR PATCH

Statistics on expressions crash

With extended statistics on an expression, repeating the same condition twice trips an assertion in the planner's row estimate code.

```
CREATE TABLE dt (a int, c bool);
INSERT INTO dt SELECT x % 10, x % 10 < 5
  FROM generate_series(1, 1000) x;
CREATE STATISTICS s (dependencies)
  ON a, (c IS TRUE) FROM dt;
ANALYZE dt;
SELECT * FROM dt WHERE a = 1
  AND (c IS TRUE) AND (c IS TRUE);   -- crash
```

Found by our extended-statistics fuzzer: random statistics objects plus WHERE clauses that reuse the same expressions.

Fix our patch, committed by David Rowley (b0392c2), back-patched to v14.

04

FIXED · OUR PATCH

NOT IN returns a row it should not

A new v19 speed-up turns NOT IN into an anti-join. It assumes no NULLs, but an implicit cast can turn a value into NULL, so a wrong row comes back.

```
-- i2t(0) returns NULL
CREATE CAST (int AS text)
  WITH FUNCTION i2t(int) AS IMPLICIT;
-- o.t holds '1',  s.i holds 0
SELECT t FROM o
WHERE t NOT IN (SELECT i FROM s);
-- expected: no rows   got: '1'
```

Found by differential checks: release 18, without the new speed-up, returns no rows; master returns '1'.

Fix our patch, committed by Richard Guo (b7eee09) for v19.

Bugs 5–6: JSON_TABLE and a query that never stops

05

FIXED · OUR PATCH · #19768

JSON_TABLE gives wrong answers

JSON_TABLE forgot to assign collations to its input, so a row comparison inside it compares zero columns: wrong answer on 17/18, crash on master.

```
SELECT * FROM JSON_TABLE(
  jsonb_build_object('k',
      ROW(1, 2) < ROW(3, 4)),
  '$' COLUMNS (a bool PATH '$.k'));

-- expected: t
-- 17, 18:   f   (wrong answer)
-- master:   server crashes
```

Found by the grammar fuzzer hit the crash on master; running the same query on release 18 showed the wrong answer.

Fix our patch, committed by Amit Langote (e405a46), back-patched to v17.

06

FIXED · OUR PATCH

A query that never stops

first_value() / nth_value() with the new IGNORE NULLS loop forever. The query ignores statement_timeout and cancel; only killing the process helps.

```
SET statement_timeout = '5s';
SELECT first_value(x) IGNORE NULLS OVER w
FROM (VALUES (1)) v(x)
WINDOW w AS (ROWS BETWEEN UNBOUNDED PRECEDING
  AND UNBOUNDED FOLLOWING EXCLUDE GROUP);

-- never returns, ignores the timeout
```

Found by the hang check: the query ran past the timeout and could not be cancelled.

Fix our patch, committed by Tatsuo Ishii (69b20fd) for v19.

All nine bugs

BUG	KIND	FOUND BY	STATUS
#19803 Parallel planner crash	crash	LLM arm	fixed
#19771 to_char() drops output	wrong result	cross-version	fixed · our patch
Statistics on expressions crash	crash	statistics fuzzer	fixed · our patch
NOT IN wrong row	wrong result	differential checks	fixed · our patch
#19768 JSON_TABLE wrong answer	wrong result, crash	grammar fuzzer	fixed · our patch
IGNORE NULLS never stops	hang	hang check	fixed · our patch
#19802 MERGE internal error	internal error	grammar fuzzer	confirmed
#19793 Range check wrong for NULL	wrong result	rewrite oracle	reported + patch
#19766 Parallel EXPLAIN crash	crash	grammar fuzzer	reported

6

fixed upstream, 5 with our patches

4

kinds of failure: crash, wrong result, internal error, hang

Writing valid SQL is the easy part; knowing the right answer is the hard part.

Thank you

Questions?

Code, reproducers and patches

github.com/PASS-IITG/cs5187-term-project-blazing-bhuvi

Venu Kummari · Bhuvan Chilukuti · Gowtham Reddy